ECE 5755 Lab 2: Tiling/Blocking, Quantization and Attention

Intel Xeon Silver 4514Y (Sapphire Rapids), gcc 13.3, Top-down level 1–2 via `toplev.py`

Name: Abiola **NetID:** ab3394

1. Implementation summary

`matmul_blocking` splits $C = AB$ into $B_I \times B_K$ tiles of A, $B_K \times B_J$ tiles of B and $B_I \times B_J$ tiles of C (outer loops i_0, k_0, j_0). Inside a tile the loops run in *i-k-j* order: `A[i][k]` is held in a register while `B[k][j..]` and `C[i][j..]` are streamed with unit stride, so every fetched cache line is fully used. The final tile is $64 \times 256 \times 32$: the B tile is $32 \times 256 \times 4\,\mathrm{B} = 32\,\mathrm{KB}$ and fits in the 48 KB L1d, and it is reused by all 64 rows of the A tile before being evicted. Edge tiles are clipped, so any shape works (tested on $131 \times 77 \cdot 77 \times 301$ against `matmul`). `matmul_quant` unpacks four 8-bit values per `unsigned int`, most significant byte first, with a shift and `&0xFF` mask (`0x01020304` $\rightarrow [1, 2, 3, 4]$ multiplies rows $4w..4w{+}3$ of B), accumulates in a 64-bit integer and casts to float. `scaled_dot_product_attention` computes $\mathrm{softmax}(QK^T/\sqrt{d})V$ row by row, subtracting the row maximum before `expf` for numerical stability. All 19 Unity tests pass, including new tests for edge tiles, the handout's quantization example, uniform attention, and empty inputs to `flatten` and `linear` (which now return `NULL`).

2. Profiling process

- **Program under test.** `perf/matmul_naive.c`, `perf/matmul_blocking.c` and `perf/matmul_blocking_ijk.c` each build two random $N \times N$ float matrices, then call one kernel `reps` times in a loop (N=256: 100, 1024: 3, 2048: 1), freeing each result, so the matmul dominates runtime. Setup alone (allocation, `rand()`, program start; measured with reps = 0) takes 0.1–0.3 s, i.e. $\leq$6% of every run except the fastest 256^2 `-O2` runs ($\sim$10%). Tile sizes are compile-time macros (`-DBLOCK_SIZE_I=...`), so one source file builds every tiled variant.
- **How it is measured.** The program is pinned to one core with `taskset -c 0`, and toplev collects Top-down counters for that core only:
  `toplev.py --core S0-C0 -l2 -v --no-desc --force-cpu spr taskset -c 0 ./prog N reps`
  Runtime comes from separate runs under `/usr/bin/time` (wall-clock *real* time), one job at a time. **Every toplev and timing measurement is repeated in 3 independent trials** (separate program runs); the figure and table report the mean, and the table also gives the standard deviation. Every program is built at the Makefile default (`-O0`, as in Lab 1) and again at `-O2`, because `-O0` hides cache effects (Section 4). Everything is scripted in `scripts/run_perf.sh`; raw outputs are in `perf_results/`.
- **Inputs.** N=256 (3 matrices = 768 KB, larger than L1 but within L2), 1024 (12 MB, larger than L2) and 2048 (48 MB, larger than L3).
- **Implementations.** Lab 1 `matmul` (ijk); a tiled version with ijk order inside the tile (my first attempt); tiled ikj with small $16 \times 64 \times 16$ tiles; tiled ikj with $64 \times 256 \times 32$ tiles (final). A sweep of 27 tile shapes (`scripts/sweep_tiles.sh`) chose the final tile.

3. Results

Reading the labels. Each row in Figure 1 and Table 2 is "N^2 *flag implementation*": N is the matrix size and *flag* the gcc optimization level. Implementations are named by the order of their three nested loops, outermost first: i is a row of A and C, j a column of B and C, and k the dimension summed over (a column of A and a row of B). All of them compute `C[i][j] += A[i][k] * B[k][j]`; only the loop order and tiling differ.

Label	Loops (outer → inner)	Tile $B_I \times B_J \times B_K$	Innermost loop walks
matmul (Lab 1, ijk)	i, j, k (no tiling)	–	down a column of B (new row every step)
tiled ijk 64x256x32	tiles i_0, j_0, k_0; in tile i, j, k	$64 \times 256 \times 32$	down a column of B, within the tile
tiled ikj 16x64x16	tiles i_0, k_0, j_0; in tile i, k, j	$16 \times 64 \times 16$	along a row of B and C (unit stride)
tiled ikj 64x256x32 (final)	tiles i_0, k_0, j_0; in tile i, k, j	$64 \times 256 \times 32$	along a row of B and C (unit stride)

Table 1. Implementations compared. The final row is the submitted `matmul_blocking`.

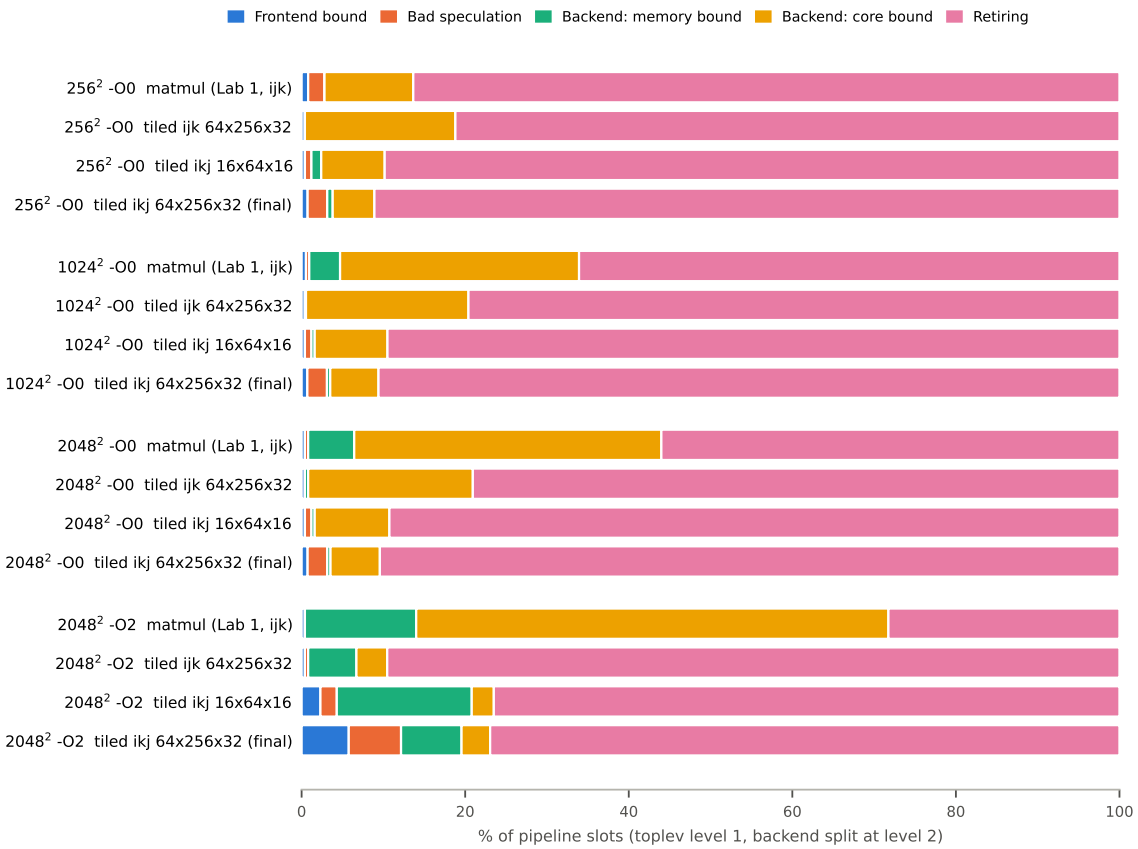


Figure 1. Top-down breakdown per implementation and input (% of pipeline slots; backend split into memory/core at level 2), mean of 3 trials. Per-segment standard deviation across trials: median 0.2, max 1.2 percentage points.

		`-O0` (Makefile default)			`-O2`		
N	Implementation	real s	s/call	speedup	real s	s/call	speedup
256	matmul (Lab 1, ijk)	6.26 ± 0.02	0.063	1.00×	1.36 ± 0.00	0.014	1.00×
256	tiled ijk 64x256x32	4.55 ± 0.00	0.045	1.38×	0.94 ± 0.01	0.009	1.44×
256	tiled ikj 16x64x16	2.93 ± 0.05	0.029	2.13×	0.78 ± 0.00	0.008	1.74×
256	tiled ikj 64x256x32 (final)	2.86 ± 0.03	0.029	2.19×	0.80 ± 0.00	0.008	1.70×
1024	matmul (Lab 1, ijk)	15.72 ± 0.01	5.239	1.00×	4.19 ± 0.01	1.396	1.00×
1024	tiled ijk 64x256x32	8.91 ± 0.01	2.969	1.76×	1.93 ± 0.01	0.644	2.17×
1024	tiled ikj 16x64x16	5.66 ± 0.06	1.886	2.78×	1.66 ± 0.00	0.553	2.52×
1024	tiled ikj 64x256x32 (final)	5.53 ± 0.02	1.842	2.84×	1.61 ± 0.00	0.537	2.60×
2048	matmul (Lab 1, ijk)	49.37 ± 0.10	49.373	1.00×	13.55 ± 0.51	13.550	1.00×
2048	tiled ijk 64x256x32	23.92 ± 0.02	23.923	2.06×	5.22 ± 0.01	5.217	2.60×
2048	tiled ikj 16x64x16	15.18 ± 0.20	15.180	3.25×	4.49 ± 0.00	4.490	3.02×
2048	tiled ikj 64x256x32 (final)	14.73 ± 0.04	14.733	3.35×	4.36 ± 0.00	4.360	3.11×

Table 2. Wall-clock time of the whole program, mean ± standard deviation over 3 trials (reps = 100 / 3 / 1 for N = 256 / 1024 / 2048); speedup relative to Lab 1 `matmul`.

4. Discussion

`matmul_blocking` vs. `matmul`. The final blocked kernel is 2.2×, 2.8× and 3.4× faster than `matmul` at N=256, 1024 and 2048 (`-O0`), and 1.7×, 2.6× and 3.1× faster at `-O2`. The gain grows with N: per call, `matmul` slows down 9.4× from 1024 to 2048 at `-O0` (9.7× at `-O2`) for 8× the work, while the blocked kernel scales by 8.0–8.1×. Its runtime therefore stays proportional to the FLOP count even after the working set no longer fits in L2 (1024) or L3 (2048).

Why `matmul` is slow (Core/Memory bound). In `matmul` the inner k loop reads `B[k][j]` down a column. Each B row is a separate allocation, so every iteration touches a new cache line (and a new 4 KB page every iteration for $N \geq 1024$) and uses only 4 of its 64 bytes. That is poor spatial locality, and it shows up as Memory_Bound rising with N (0%→3.8%→5.6% at `-O0`; 6.3%→13.6% from 1024 to 2048 at `-O2`). The larger cost is Core_Bound (29–38% at `-O0`

for $N \geq 1024$, 58% at `-O2`). Every iteration adds into the same accumulator (`C[i][j]` kept in memory at `-O0`, `sum` in a register at `-O2`), forming a loop-carried dependence chain of FP adds over all N values of k. The out-of-order core cannot overlap iterations, so execution ports sit idle while each add waits for the previous one, and the column loads that miss in L1 lengthen each step of the chain.

Why tiling + *ikj* helps. In the *ikj* tile loop, consecutive iterations write different `C[i][j]` elements, so they are independent and the core issues them back-to-back. Core_Bound drops to 5–6% and Retiring rises to 90–91% at `-O0`, for every N. B and C are accessed with unit stride (each line fully used, hardware prefetcher friendly), and the 32 KB B tile is reused by 64 rows of A from L1 before eviction. Memory_Bound therefore stays under 1% at `-O0`, and at `-O2` is 7.4% at N=2048 versus 13.6% for `matmul`, even though the matrices are larger than L3.

Comparing implementations. *At `-O0`, loop order matters more than tiling.* The tiled-*ijk* variant has the same tiles but keeps the column walk and the dependent accumulation chain: it is still 20% Core_Bound and only 1.4–2.1× faster than `matmul`. At `-O2` the gap narrows. gcc keeps tiled-*ijk*'s partial sum in a register and its chain is only B_K=32 adds long, so it is 89–90% Retiring and 2.2–2.6× faster than `matmul`. It is nevertheless 17–20% slower than the final kernel (5.22 vs. 4.36 s at N=2048): `objdump` shows 9 instructions per multiply-add (it reloads the row pointer `B[k]` and has two loop-exit branches) versus 7 for *ikj*. High Retiring means the slots do useful work, not that little work is needed. *Tile size matters little here.* At `-O0` the small $16 \times 64 \times 16$ and large $64 \times 256 \times 32$ tiles are within 3% of each other, and a 27-point tile sweep at N=1024 spanned only 1.83–1.97 s, because unoptimized code spends 24 instructions per multiply-add (from `objdump`) on stack traffic. At `-O2` the small tile is 3% slower for $N \geq 1024$ and is 16.5% Memory_Bound versus 7.4%: each B tile is reused by only 16 rows of A, so B is streamed from L2/L3 four times as often. At N=256, where everything fits in L2, the small tile is marginally faster (0.78 vs. 0.80 s).

Remaining bottleneck. At `-O2` the final kernel is 77% Retiring, with the rest split between Memory_Bound (7%), Bad_Speculation (6%), Frontend_Bound (6%) and Core_Bound (3%). Its scalar inner loop does one multiply-add per 7 instructions, and `-fopt-info-vec` confirms gcc does not vectorize it at `-O2`; SIMD (AVX-512, 16 floats per instruction) would be the next step.

Effect of input size. At N=256 all three matrices fit in L2, so `matmul` is already 86% Retiring at `-O0` and blocking gains less (2.2× at `-O0`, 1.7× at `-O2`). Larger inputs make the baseline's poor locality costlier, while the blocked kernel's profile stays flat.

5. Issues and debugging

- **Quantization byte order.** My first `matmul_quant` extracted the least significant byte as element 0. The provided test uses symmetric words (`0x01010101`), so it passed anyway. A new test built from the handout's `0x01020304` example returned 12340 instead of 43210, which exposed the reversed order. I fixed the shift to $(3 - e) \cdot 8$.
- **Ineffective first blocking attempt.** My first tiled kernel (*ijk* inside tiles) was only 1.4× faster at `-O0`; its 20% Core_Bound pointed to the accumulation chain and led to the *ikj* rewrite.
- **toplev measured nothing on the login node** (`perf_event_paranoid`=4). Moving to the Slurm node fixed it. The Makefile's toplev command originally profiled `S0-C0` without pinning the program, so I added `taskset -c 0`. Jobs are limited to 5 minutes and all pin to CPU 0, so data collection was split into sequential `srun` jobs.
- **Code alignment skewed my first `-O2` data.** My first `-O2` profiles showed the tiled kernels 25–43% Frontend_Bound. Repeated trials agreed with each other but not with the first: Frontend_Bound fell below 6% and the tiled kernels ran 16–36% faster. `objdump` showed identical instructions; a small edit to `main()` had moved `matmul_blocking` 32 bytes, so the hot loop that had straddled a 64-byte boundary (0x2938–0x2954) now sits inside one (0x2958–0x2974). The frontend fetches decoded uops in 64-byte windows, so the old layout most likely needed two fetches per iteration. All `-O2` results here are 3 trials of the current binaries (old data in `perf_results/old_layout_O2/`).
- **Noise.** Repeating the tile sweep three times showed ∼5% run-to-run variation, larger than most differences between tile shapes. I therefore chose the final tile on cache-capacity reasoning among the statistically tied fastest shapes.
- **Attention.** The score buffer was a variable-length array on the stack. I replaced it with `malloc` so long sequences cannot overflow the stack.

6. AI usage statement

The code implementation for this lab (`matmul_blocking`, `matmul_quant`, `scaled_dot_product_attention` and the accompanying tests) was developed with the assistance of an AI coding assistant (Claude Code, Anthropic). All code was compiled, tested and profiled on the course machines, and the results reported here are from those runs.